End-to-End Encryption and Security Architecture for Cloud-Native Applications

Zach Kelling

Hanzo AI

2019

Abstract

We describe the encryption and security architecture developed at Hanzo Industries between 2018 and 2019, comprising four components: Age-based file encryption using X25519 key agreement (`hanzo/age`, May 2019), an encrypted message queue (`hanzo/mq`, December 2018, 4,900 commits), per-tenant storage encryption for the S3-compatible object store, and a key management architecture that serves as the precursor to the Hanzo KMS. Together, these components provide end-to-end encryption for data at rest, data in transit within the message queue, and data in storage. The security primitives described here subsequently evolved into the post-quantum encryption stack deployed on the Lux network.

1 Introduction

Cloud-native applications handle sensitive data across multiple services, storage backends, and communication channels. Encryption must be applied consistently at every layer: files at rest, messages in transit between services, objects in storage, and secrets in configuration. Relying on transport-layer encryption (TLS) alone is insufficient—data is decrypted at each service boundary, creating exposure windows.

The Hanzo security architecture addresses this through defense-in-depth encryption:

1. **File encryption** (May 2019): Age-based encryption for configuration files, backups, and secrets at rest.
2. **Message encryption** (December 2018): Per-message encryption within the message queue.
3. **Storage encryption**: Per-tenant encryption of objects in the S3-compatible store.
4. **Key management**: Centralized key lifecycle management.

2 Age Encryption

2.1 Design

The `hanzo/age` module (May 2019) implements file encryption using the Age format [1]. Age uses X25519 key agreement with ChaCha20-Poly1305 symmetric encryption:

$$K_{\text{shared}} = \text{X25519}(sk_{\text{sender}}, pk_{\text{recipient}}) \tag{1}$$

$$K_{\text{file}} \xleftarrow{\$} \{0,1\}^{256} \tag{2}$$

$$C_{\text{header}} = \text{ChaCha20-Poly1305}_{K_{\text{shared}}}(K_{\text{file}}) \tag{3}$$

$$C_{\text{body}} = \text{ChaCha20-Poly1305}_{K_{\text{file}}}(P) \tag{4}$$

where sk_{sender} is the sender's private key, $pk_{\text{recipient}}$ is the recipient's public key, K_{file} is a per-file symmetric key, P is the plaintext, and C_{body} is the ciphertext.

2.2 Key Format

Public keys are encoded as Bech32 strings with a human-readable prefix:

Listing 1: Age key pair format.

```
# Public key
age1ql3z7hjy54pw3hyww5ayyfg7zqgvc7w3j2elw8zmrj2kg5sfn9aqmcac8p

# Private key (stored in KMS, never on disk)
AGE-SECRET-KEY-1QFWET6A35FH2XNNWCJLZ5VQCGY3P46S...
```

2.3 Use Cases

Age encryption is used for:

- **Secret files**: Encrypting configuration files containing credentials before storing them in version control or backup storage.
- **Database backups**: Encrypting PostgreSQL and KV store backups before uploading to object storage.
- **Compliance exports**: Encrypting data exports for regulatory compliance.

3 Encrypted Message Queue

3.1 Architecture

The Hanzo message queue (`hanzo/mq`, December 2018, 4,900 commits) extends the PubSub layer with per-message encryption. Messages containing sensitive data are encrypted before publication and decrypted by authorized subscribers.

3.2 Envelope Encryption

Messages use envelope encryption: each message is encrypted with a unique data encryption key (DEK), and the DEK is encrypted with the recipient's key encryption key (KEK):

$$DEK_m \xleftarrow{\$} \{0,1\}^{256} \tag{5}$$

$$C_m = \text{AES-256-GCM}_{DEK_m}(M) \tag{6}$$

$$E_m = \text{AES-256-GCM}_{KEK_r}(DEK_m) \tag{7}$$

where M is the plaintext message, C_m the ciphertext, DEK_m the per-message data encryption key, and KEK_r the recipient's key encryption key.

The published message contains $(E_m, \text{nonce}, C_m, \text{tag})$. Only subscribers with access to KEK_r can decrypt DEK_m and subsequently M.

3.3 Key Rotation

KEKs are rotated on a configurable schedule (default: 90 days). During rotation:

1. A new KEK is generated and distributed to authorized subscribers.
2. New messages are encrypted with the new KEK.

3. The old KEK is retained for a grace period to allow in-flight message decryption.
4. After the grace period, the old KEK is archived to the key management layer.

3.4 Message Queue Performance

Table 1: Encryption overhead on message throughput.

Mode	Messages/s	Overhead
Plaintext PubSub	3,500,000	—
Encrypted MQ (256B)	2,800,000	20%
Encrypted MQ (4KB)	1,900,000	46%
Encrypted MQ (64KB)	420,000	88%

The overhead is dominated by the symmetric encryption of the message body. The envelope key operation adds negligible latency (sub-microsecond).

4 Per-Tenant Storage Encryption

4.1 Architecture

The S3-compatible object storage layer encrypts all objects at rest using per-tenant keys. The encryption is transparent to clients: objects are encrypted on write and decrypted on read by the storage service.

4.2 Key Hierarchy

A three-tier key hierarchy provides isolation and rotation:

1. **Master Key (MK)**: A single master key stored in a hardware security module (HSM) or software KMS. Never leaves the key management boundary.
2. **Tenant Key (TK)**: One per tenant, encrypted by the MK. Stored alongside the tenant metadata.
3. **Object Key (OK)**: One per object, encrypted by the TK. Stored in the object metadata.

$$OK_o \xleftarrow{\$} \{0,1\}^{256}, \quad C_o = \text{AES-256-GCM}_{OK_o}(O) \tag{8}$$

$$E_{OK} = \text{AES-256-GCM}_{TK_t}(OK_o), \quad E_{TK} = \text{AES-256-GCM}_{MK}(TK_t) \tag{9}$$

4.3 Tenant Isolation Guarantee

Compromising a single tenant key TK_t exposes only that tenant's objects. The master key and all other tenant keys remain unaffected. Tenant key rotation re-encrypts only the tenant key itself (not the underlying objects), making rotation a constant-time operation:

$$E'_{TK} = \text{AES-256-GCM}_{MK}(TK'_t), \quad \forall\, o : E'_{OK_o} = \text{AES-256-GCM}_{TK'_t}(OK_o) \tag{10}$$

In practice, lazy re-encryption of object keys is performed during object access, amortizing the cost over normal read/write operations.

5 Key Management Architecture

5.1 Design Principles

The key management layer follows three principles:

1. **Separation of duties**: The system that encrypts data does not have direct access to the master key. Encryption operations go through the KMS API.
2. **Audit trail**: Every key operation (creation, rotation, access, deletion) is logged immutably.
3. **Least privilege**: Services are granted access only to the specific keys required for their function.

5.2 KMS API

The key management API provides four operations:

Listing 2: Key management interface.

```go
type KMS interface {
    // Create a new encryption key
    CreateKey(ctx context.Context,
        name string) (*Key, error)

    // Encrypt data with a named key
    Encrypt(ctx context.Context,
        keyName string,
        plaintext []byte) ([]byte, error)

    // Decrypt data with a named key
    Decrypt(ctx context.Context,
        keyName string,
        ciphertext []byte) ([]byte, error)

    // Rotate a key (creates new version)
    RotateKey(ctx context.Context,
        keyName string) (*Key, error)
}
```

5.3 Access Control

Access to keys is controlled by IAM policies. Each key has an access control list specifying which service identities may encrypt, decrypt, or administer the key:

Table 2: Key access policy example.

Service	Key	Permissions
storage-service	tenant-keys/*	encrypt, decrypt
backup-service	backup-key	encrypt only
restore-service	backup-key	decrypt only
kms-admin	*	admin

6 Threat Model

The encryption architecture defends against the following threats:

- **Storage breach**: An attacker gaining read access to the storage backend obtains only ciphertext. Without the tenant key, objects cannot be decrypted.
- **Message interception**: An attacker capturing messages from the queue obtains only encrypted payloads. Without the recipient KEK, messages cannot be decrypted.
- **Insider threat**: A compromised service can only access keys granted to its identity. Separation of encrypt/decrypt permissions limits blast radius.
- **Key compromise**: Per-tenant and per-object key hierarchy limits the scope of any single key compromise.

7 Evolution Toward Post-Quantum Security

The X25519 key agreement and AES-256-GCM symmetric encryption provide strong security against classical adversaries. However, the emergence of large-scale quantum computing threatens the security of elliptic curve key agreement. The key management architecture was designed with algorithm agility: the KMS abstraction allows replacement of the underlying cryptographic primitives without changing the API surface.

This design decision enabled the subsequent adoption of post-quantum key encapsulation mechanisms (KEMs) on the Lux network, where Kyber/ML-KEM replaced X25519 for key agreement while maintaining the same envelope encryption pattern and key hierarchy.

8 Conclusion

The Hanzo encryption and security architecture provides defense-in-depth protection for data at rest, in transit within the message queue, and in object storage. Age encryption handles file-level encryption with X25519 key agreement. The encrypted message queue extends PubSub with per-message envelope encryption. Per-tenant storage encryption with a three-tier key hierarchy ensures tenant isolation. The key management API centralizes key lifecycle operations with IAM-controlled access policies. These primitives, developed between 2018 and 2019, formed the security foundation that subsequently evolved into the post-quantum encryption stack deployed on the Lux network.

References

[1] F. Valsorda. age: A Simple, Modern, and Secure Encryption Tool. `https://age-encryption.org`, 2019.

[2] A. Langley, M. Hamburg, and S. Turner. Elliptic Curves for Security. RFC 7748, IETF, 2016.

[3] D. McGrew. An Interface and Algorithms for Authenticated Encryption. RFC 5116, IETF, 2008.

[4] NIST. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM). SP 800-38D, 2007.

[5] D. J. Bernstein. Curve25519: New Diffie-Hellman speed records. *Lecture Notes in Computer Science*, 3958:207–228, 2006.